Лабораторная работа №3. Транспонирование матрицы и исследование локальности памяти.

1. Характеристики системы: процессор: 11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz; Ядра: 4 физических, 8 логических (Hyper-Threading); Кэш L2: 5 120 КБ (5 МБ); Кэш L3: 8 192 КБ (8 МБ); GCC: 14.2.0.

2. Таблицы с результатами экспериментов.
Эксперимент 1. Влияние порядка обхода (1 поток):

N	Время rows, с	Время cols, с	BW rows, ГБ/с	BW cols, ГБ/с
256	<0.001	<0.001	—	—
512	0.0010	0.0010	4.19	4.19
1024	0.0060	0.0060	2.80	2.80
2048	0.0350	0.0360	1.92	1.86
3500	0.0820	0.0890	2.39	2.20

Эксперимент 2. Влияние размера блока (1 поток, N=3500):

b	Время, с	BW, ГБ/с
4	0.0220	8.91
8	0.0180	10.89
16	**0.0180**	**10.89**
32	**0.0180**	**10.89**
64	0.0190	10.32
128	0.0200	9.80

Эксперимент 3. Влияние числа потоков (N=3500, b=32):

Потоки	Время rows_omp, с	S rows_omp	Время blocked_omp, с	S blocked_omp
1	0.0800	1.00	0.0200	1.00
2	0.0430	1.86	0.0100	2.00
4	0.0220	3.64	0.0090	2.22
8	0.0230	3.48	0.0090	2.22
16	0.0230	3.48	0.0070	2.86

Эксперимент 4. False sharing (8 потоков, N=3500):

Стратегия	Время, с	BW, ГБ/с
static	0.0220	8.91
static, 1	0.0330	5.94
static, 8	0.0270	7.26

3. Вывод о проделанной работе.
При малых N (256, 512) разница между обходом по строкам и по столбцам незаметна, так как матрица целиком помещается в кэш. При N = 3500 транспонирование по столбцам становится немного медленнее из-за прыжков по памяти, хотя аппаратный префетчер процессора частично сглаживает разницу. Найденный оптимальный размер блока b* = 16–32, что соответствует размеру кэш-линии и позволяет блоку целиком поместиться в L1/L2 кэш процессора. Ускорение транспонирования ограничено сильнее, чем в ЛР2, потому что задача ограничена пропускной способностью памяти (memory-bound), а не вычислениями: четыре физических ядра уже насыщают шину памяти, и добавление потоков не даёт линейного ускорения. В эксперименте с False sharing видно, что стратегия static, 1 работает в полтора раза медленнее static: при малом chunk соседние элементы одной кэш-линии обрабатываются разными потоками, что заставляет процессор постоянно инвалидировать кэш. Блочное транспонирование с b=32 оказалось самым быстрым, так как сочетает локальность данных и хороший параллелизм.